Failure (accidentally blocking the event loop) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exercise is really about one substitution — `time.sleep` in place of `await asyncio.sleep` — and what it does to a `gather` call that should otherwise run concurrently.

You'll write two coroutine functions, both sleeping the same duration so the comparison is clean, run together with `asyncio.gather(blocking_task(), good_task())` — blocking one **first**, timed with `time.perf_counter()` around the whole call. If concurrency were actually happening, the total time should be roughly the max of the two (about 3 seconds). Predict, before running it, what you think the actual total will be instead.

The exact pieces:

- **`time.sleep(3)` inside an `async def` function** — still syntactically legal Python; nothing raises an error. The function is still a coroutine because of `async def`, but since it never hits an `await`, the event loop has no opportunity to pause it and run something else.
- **`await asyncio.sleep(3)`** — genuinely returns control to the event loop, which is exactly what lets a *second* coroutine run during that wait.
- **The comparison that proves it:** two "good" coroutines run together finish in ~3 seconds (the max). One "good" and one "blocking" finish in ~6 seconds (the sum) — the blocking one froze everything else, including the good one, until it was done.
- **Order matters:** put `blocking_task()` first. If `good_task()` goes first, its `asyncio.sleep` clock is already running when the loop freezes, the waits still overlap, and you'd see ~3 seconds — the bug hides.
- **No error, no warning:** this is what makes the bug sneaky — the code runs, gives a correct final answer, and the only symptom is that it took twice as long as it should have.

Think about where this actually happens in real code — nobody writes `time.sleep(3)` inside an `async def` function on purpose in production. It happens by accident, almost always the same way: someone reaches for a library that only ships a synchronous version — a database driver, a file-parsing library, `requests` instead of `httpx` / `aiohttp` — and calls it from inside an `async def` function without realizing that call blocks the whole event loop exactly like `time.sleep` does here, just less obviously, since it doesn't have "sleep" in its name to give it away.

The real design question isn't just "did I accidentally block the loop" — it's "what do I do when the work I need genuinely has no async version, and I can't just rewrite the library myself?"

The extra piece that answers that question:

- **`asyncio.to_thread(...)`** (or the lower-level `loop.run_in_executor(...)`) runs a normal, blocking, synchronous function in a separate thread, and gives you back something you `await` — so the event loop stays free to run other coroutines while that blocking call happens somewhere else. `result = await asyncio.to_thread(some_blocking_function, arg1, arg2)` is the fix any time you're stuck calling code that was never written to be async.

Hint 2 — The plan, and almost the whole thing

```
import asyncio
import time

define:
    async def good_task() -> str:
        await asyncio.sleep(3)
        return "good"

define:
    async def blocking_task() -> str:
        time.sleep(3)             # the bug: no await, freezes the whole loop
        return "blocking"

define:
    async def time_it(label: str, coro_a, coro_b) -> None:
        start = time.perf_counter()
        results = await asyncio.gather(coro_a, coro_b)
        elapsed = time.perf_counter() - start
        print(f"{label} took {elapsed:.2f}s -> {results}")

define:
    async def main() -> None:
        await time_it("two good tasks:", good_task(), good_task())
        await time_it("blocking + good:", blocking_task(), good_task())
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Fill in `main()` and the entry point yourself, then compare against the Solution.

Once that's working, fix it without touching the blocking call itself:

```
make a fixed version of blocking_task that uses asyncio.to_thread
instead of calling time.sleep directly:

async def fixed_task():
    await asyncio.to_thread(time.sleep, 3)   # still calls the real
                                               # blocking function, but
                                               # off the loop's thread
    return "fixed"

time this:
    run good_task and fixed_task together with gather
    -> expect around 3 seconds again, even though fixed_task still
       calls time.sleep internally
```

```
# blocking_event_loop_practice.py
async def fixed_task() -> str:
    await asyncio.to_thread(time.sleep, 3)
    return "fixed"

async def main_fixed() -> None:
    await time_it("fixed + good (to_thread):", fixed_task(), good_task())
```

Add this third comparison to your `main()` from the Basic version, run all 3, and confirm `"fixed + good"` comes back close to 3 seconds — not 6, like `"blocking + good"` did — then compare against the Solution.

Full solution: Show me the solution